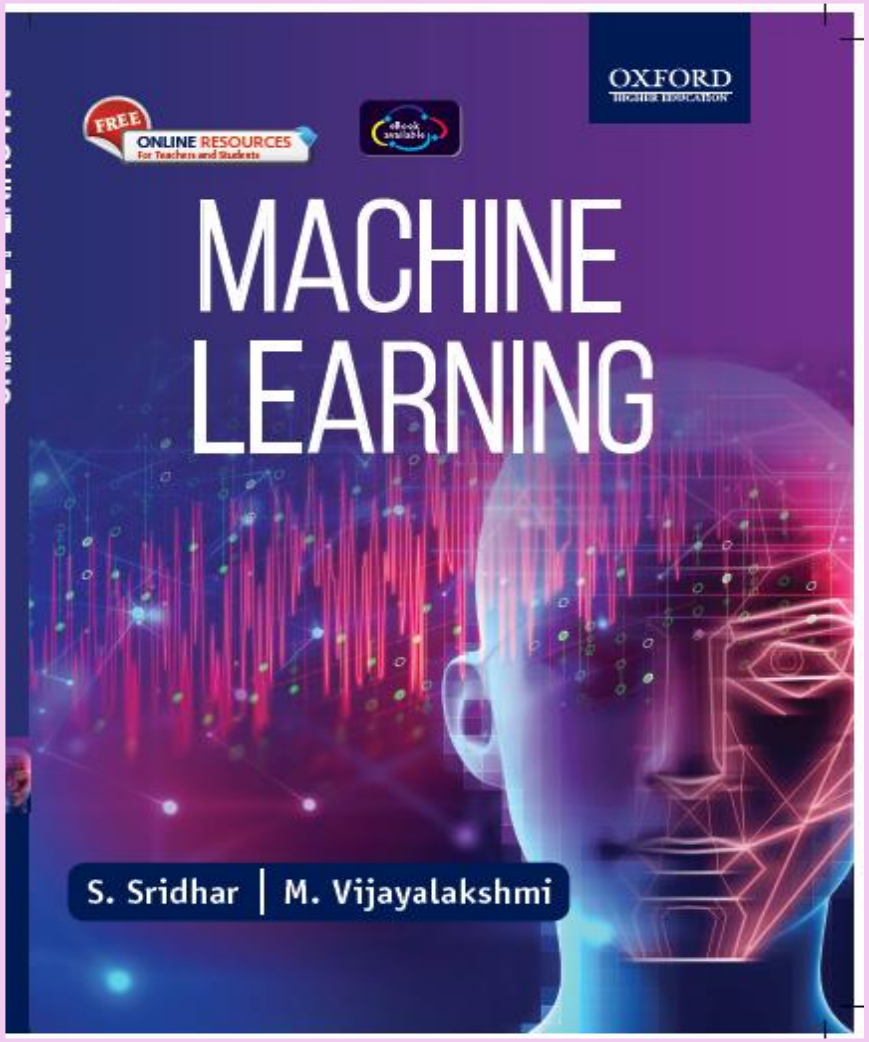


OXFORD
UNIVERSITY PRESS

Machine Learning

S. Sridhar and M. Vijayalakshmi

© Oxford University Press 2021. All rights reserved

Chapter 16

Deep Learning

© Oxford University Press 2021. All rights reserved

Deep Neural Network

If a neural network has only one hidden layer, it is called a shallow network but in the presence of two or more hidden layers, it is referred to as a deep neural network.

© Oxford University Press 2021. All rights reserved

Neural Network Algorithm

Algorithm 16.1: Neural Network

Step 1: Initialize the weights.

Step 2: Read the input vector.

Step 3: Get the output. Compute the error as the difference between the actual and expected outputs.

Step 4: Adjust the weights to reduce the computed error.

Step 5: Repeat the steps till all training data is used.

© Oxford University Press 2021. All rights reserved

Error Measure

Mean Square Error (MSE)

$$J = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y)^2$$

Likelihood Loss This measure is used for classification. It is computed as multiplication of the predicted probabilities of the inputs. If two output values or 1 and 0 are assumed, then for 1, the predicted probability p is taken as it is but for 0, the probability is $(1-p)$.

Log Loss or Cross Entropy Loss This is same as the likelihood loss, but involves logarithms. It is computed as:

$$E = -(y\log(p) + (1-y)\log(1-p)) \quad (16.3)$$

if there are two classes. If there are more classes ($M > 2$) then, it be computed as:

$$-\sum_{c=1}^{M} y_{o,c}\log(p_{o,c}) \quad (16.4)$$

Here, M is the class and p is the predicted probability of observation o of the class c.

© Oxford University Press 2021. All rights reserved

Gradient Descent

Gradient Descent Gradient Descent (GD) is one of the popular algorithms for optimization. It links the loss functions and weights of the deep neural networks. What is a gradient? A small change in weights is called a gradient. The GD algorithm changes the weight a little and measures the loss. Using the concept of gradients (partial derivatives that measure changes), the algorithm changes the weights along the suggested gradient directions. This process is continued till loss functions become low.

The weight update is calculated using the delta rule as in shallow neural networks as:

$$w_{ij} = w_{ij} + \alpha e_i x_j \quad (16.5)$$

Where, α is the learning rate, e_i is the error and w_{ij} is the weight between neurons x_i and x_j.

© Oxford University Press 2021. All rights reserved

Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) – SGD computes the error for every training data and then the weights are adjusted immediately. If the training data is 200, then 200 times the weights are adjusted. Therefore, the performance of the system is low during the training period. The weight update in SGD is computed as:

$$w_{ij} = w_{ij} + \alpha \delta_i x_j \tag{16.7}$$

Here, δ_i is the derivative of the activation function, α is the learning rate, e_i is the error and w_{ij} is the weight between neurons x_i and x_j. Since performance of SGD is low, batch mechanisms are popular.

© Oxford University Press 2021. All rights reserved

Batch Mode

Batch Mode – In batch mode, the error for all training data is computed. Then, the average of the weight update is computed and is used for adjusting the weights. In this mode, the training data and update are used only once. The formula for computing the update is:

$$w_{ij}(k) = w_{ij} + \frac{1}{N}\sum_{i=1}^{k}\Delta w_{ij}(k) \tag{16.8}$$

Here, N is the total number of training data and $\Delta w_{ij}(k)$ is the weight update of the kth sample of the training data.

© Oxford University Press 2021. All rights reserved

Mini-Batch

Mini-Batch – Mini-Batch mode chooses a small part of the training dataset. Here, the weight updates are computed for the selected data and the average weight update is computed and used. For example, a training data of 200 samples can be divided into batches of 40 samples. So, 200/40 = 5, times the weight adjustment is done.

© Oxford University Press 2021. All rights reserved

Other Techniques

AdaGrad – In AdaGrad, learning rate is dependent on the individual features. It varies according to the features.

RMSProp – RMSProp is a special variant of AdaGrad where only gradients of the fixed window are considered instead of all gradients.

Adaptive Moment Estimation (ADAM) – This optimizer uses past gradients to compute new gradients. It adds fractions of the old gradients to get the current gradient. This is also called as momentum.

© Oxford University Press 2021. All rights reserved

Regularization Techniques

Dropout Regularization - Dropout is a technique used to solve the problem of overfitting.

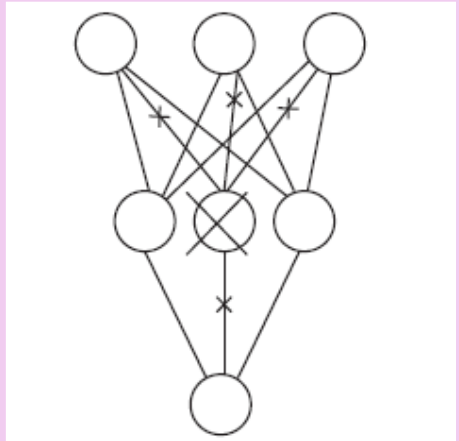


Figure 16.1: Illustration of Dropout Scheme

© Oxford University Press 2021. All rights reserved

Data Augmentation

Data Augmentation – The aim of data augmentation is to increase the dataset by increasing the number of images. The increase in the number is carried out with the help of transformations such as translation, rotation and scaling. Translation means shifting the image by certain pixels in horizontal or vertical directions. Rotation of the object is done by a certain degree in clockwise or counter-clockwise direction.

© Oxford University Press 2021. All rights reserved

Other Regularization Techniques

Early Stopping - The aim is to avoid the problem of overfitting by training the model just enough to capture the basic structure of the dataset. The processing of stopping early is one solution of solving the problem of overfitting.

Adding Noise - The focus is to increase the dataset by injecting the noise into the existing dataset. The idea is to add noise to the dataset. This avoids the problem of overfitting as the model may not perform well for the dataset. Instead, it may increase the chance of capturing the underlying structure of the dataset.

© Oxford University Press 2021. All rights reserved

Convolutional Neural Networks

The CNN architecture is given in Figure 16.2.

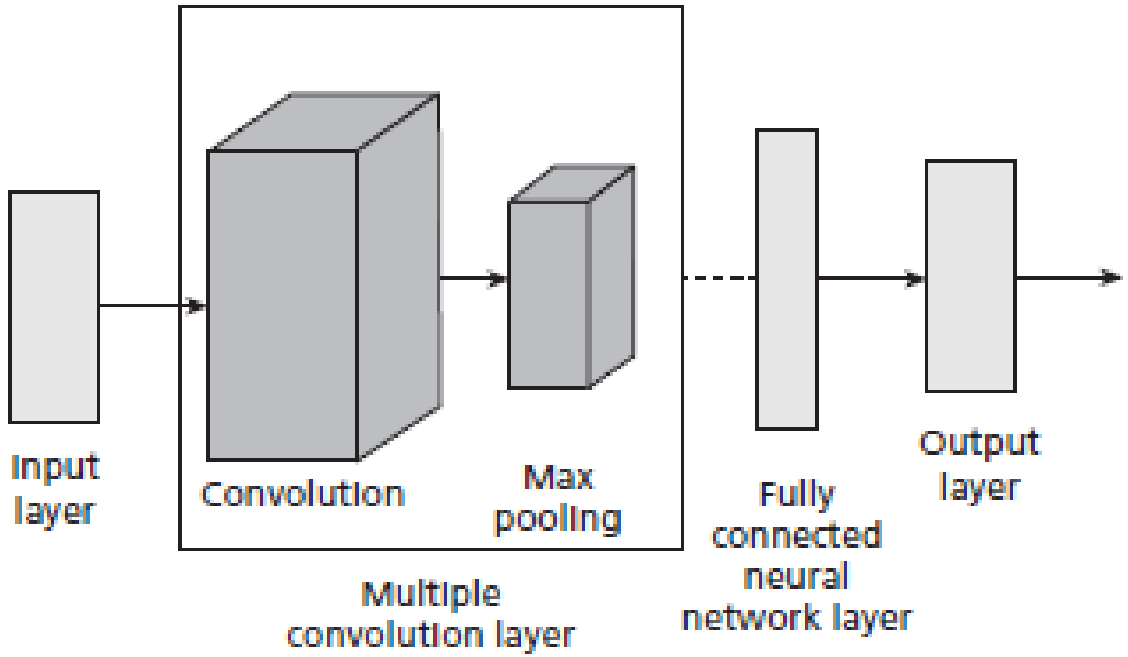


Figure 16.2: Design of a Convolutional Neural Network

© Oxford University Press 2021. All rights reserved

CNN Implementation

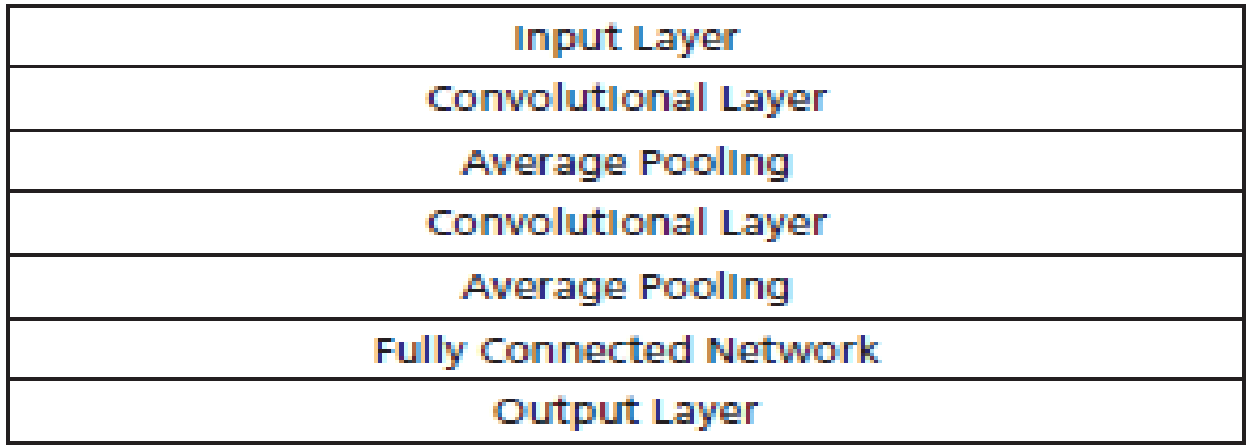


Figure 16.3: Implementation Detail of a Sample Convolutional Network

© Oxford University Press 2021. All rights reserved

Convolution Algorithm

Algorithm 16.2: Convolution

Step 1: Superimpose a filter on the image.
Step 2: Perform element-wise multiplication of the weights of filters and image.
Step 3: Sum the element-wise products to produce the output value in the resultant image.
Step 4: Repeat the steps 1-3 for all the locations of the image.

© Oxford University Press 2021. All rights reserved

Parameters

Number of Filters - Choosing the number of filters is an important criterion, as filters can learn features. But more filters add to the complexity of the operation. The number of filters is called depth.

Filter Size - Filters are also known as kernels or receptive fields. Traditionally, a 3×3 filter is used. The other sizes are 5×5 or 7×7. Optimal size of the filters is important in learning features. If the kernel size is too high, then the complexity increases as more computations are required due to an increase in the competing features. On the contrary, if the kernel size is too low, the kernel may not be able to learn any useful features.

© Oxford University Press 2021. All rights reserved

Stride

Example 16.1: Illustrate the use of Stride with input data {1, 3, 3, 2, 1, 0, 1} and filter {1, 1, 1}.

Solution: The convolution operation with the stride of 1 is shown in Figure 16.4. The convolution operation with the stride of 2 is shown in Figure 16.5. It can be observed that with the increase in stride, the dimension of the output decreases.

The logic of stride can be extended for 2D too. The following Figure 16.6 illustrates the usage of stride for two dimensions.

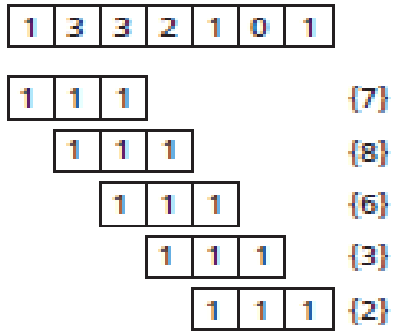


Figure 16.4: Convolution with Stride 1

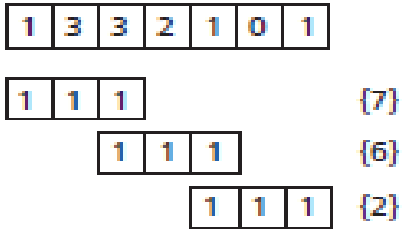


Figure 16.5: Convolution with Stride 2

© Oxford University Press 2021. All rights reserved

Stride

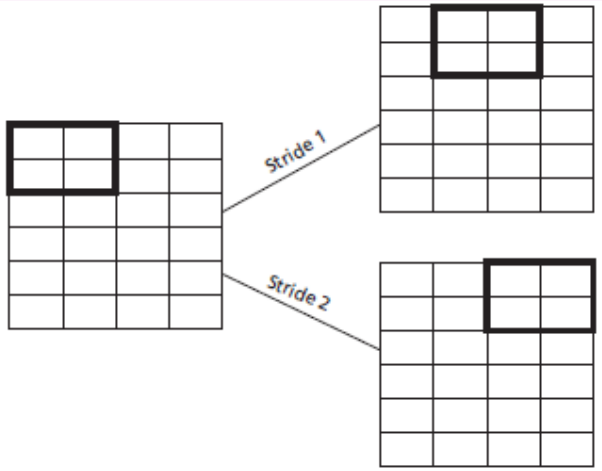


Figure 16.6: Positions of Masks After Stride with 1 and 2

© Oxford University Press 2021. All rights reserved

Padding

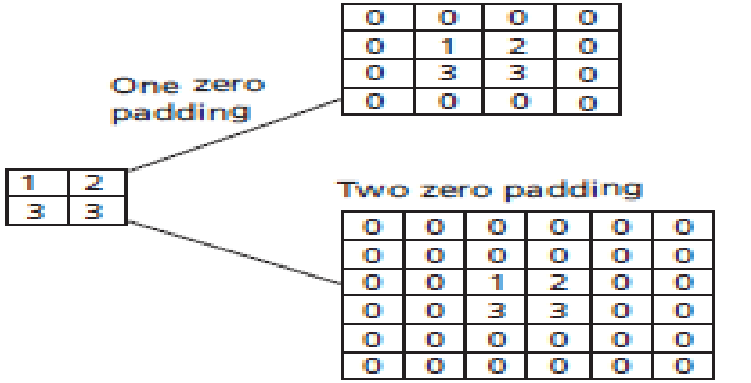


Figure 16.7: Illustration of Padding

(16.9)

© Oxford University Press 2021. All rights reserved

Activation Map

$$\text{Activation map or Feature map} = \frac{D - F + 2P}{S} + 1 \tag{16.9}$$

Here, D is the dimension of the image, F is the size of the filter, P is the amount of padding and S is the stride length.

Example 16.2: Let us assume the input size is 28×28. If the filter size is 5×5, stride is 1 and padding is zero, what is the size of the feature map?

Solution: Using Eq. (16.9), one can observe that $D = 28$, $F = 5$, $S = 1$ and $P = 0$. Therefore,

$$\text{Activation map or Feature map} = \frac{28 - 5 + 2 \times 0}{1} + 1 = 24$$

The output activation or feature map size would be 24×24.

© Oxford University Press 2021. All rights reserved

Example

Example 16.3: Consider the following data:

1	2	3	4
5	7	8	9
10	11	12	11
8	6	4	3

and mask

1	0
0	1

Show the results of the convolution operation.

Solution: The convolution obtained is shown in Figure 16.8.

1	2	3	4
5	7	8	9
10	11	12	11
8	6	4	3

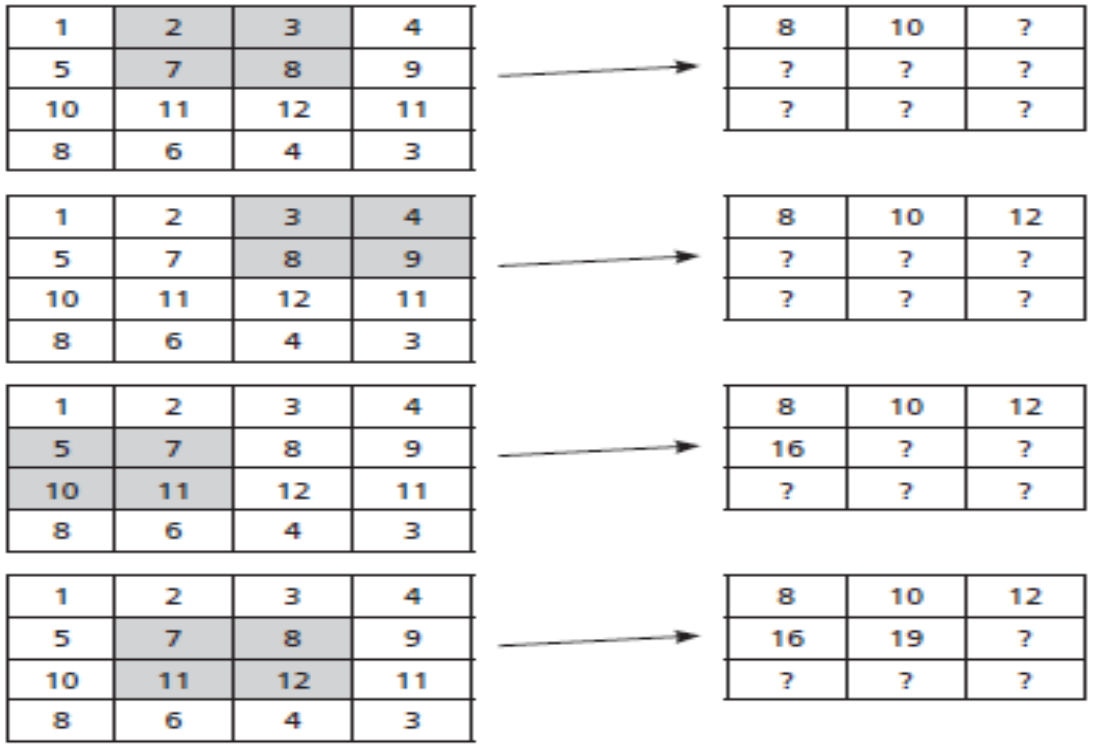


8	?	?
?	?	?
?	?	?

© Oxford University Press 2021. All rights reserved

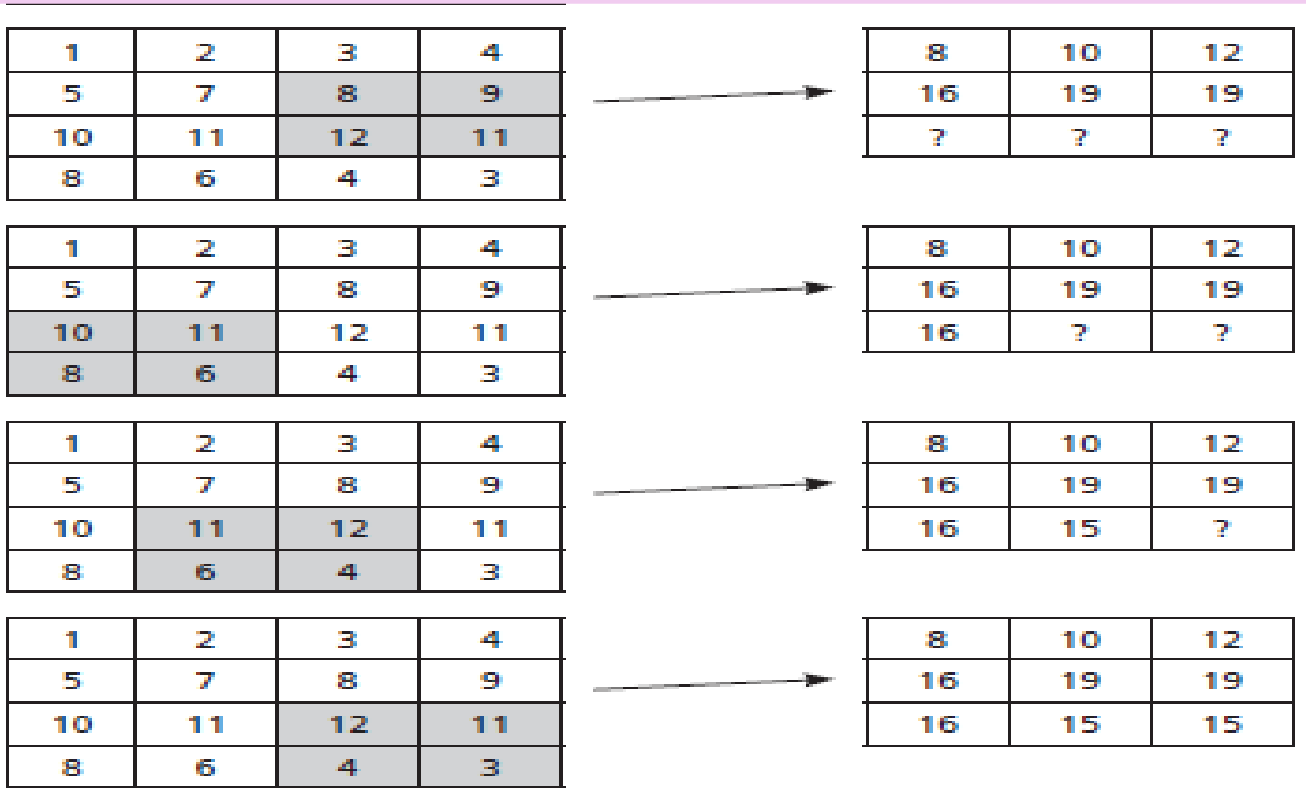
Example

This value is obtained by multiplying the elements of the image and the mask. The mask is moved into the next position.



© Oxford University Press 2021. All rights reserved

Example



1	2	3	4
5	7	8	9
10	11	12	11
8	6	4	3

8	10	12
16	19	19
?	?	?

1	2	3	4
5	7	8	9
10	11	12	11
8	6	4	3

8	10	12
16	19	19
16	?	?

1	2	3	4
5	7	8	9
10	11	12	11
8	6	4	3

8	10	12
16	19	19
16	15	?

1	2	3	4
5	7	8	9
10	11	12	11
8	6	4	3

8	10	12
16	19	19
16	15	15

Figure 16.8: Stages of Convolution

© Oxford University Press 2021. All rights reserved

Steady State Selection

This layer is also known as down sampling layer. Its main purpose is to reduce the spatial dimension of the input. This helps in two ways.

1. Reduces computational cost by reducing the weights
2. By reducing the weights, it helps to solve the overfitting problem

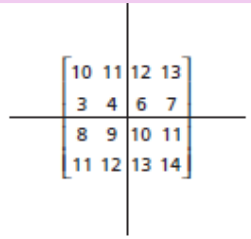


Figure 16.9: Sample Image

7	10
10	12

16.9(a): Average Pooling

11	13
12	14

Figure 16.10: Image for Max Pooling

© Oxford University Press 2021. All rights reserved

Drop-out Layer

Dropout Layers

Dropout solves the problem of overfitting. It makes a random set of activations to zero. This forces the neural network to be redundant. This ensures that the output is not too much fitted into the input.

© Oxford University Press 2021. All rights reserved

Transfer Learning

16.5 TRANSFER LEARNING

Multiple images are sometimes required to implement a solution for a given problem. Deep learning requires a greater number of training images. But the images required may be like the existing dataset.

Transfer learning refers to the concept of taking the predefined training model, the weights and parameters and fine tuning to the given dataset. It is done by removing the last layer of the predefined model using the classifier of the present problem. For example, ImageNet has predefined data of 14 million images and over thousand classes. The knowledge gained can be used by retaining the weights and parameters as such and the last layer is removed with the image classifier of the given problem.

The advantage of this setup is that the network need not be trained again. Hence, the existing weights and parameters can be used as it is. This is called knowledge transfer or transfer learning.

© Oxford University Press 2021. All rights reserved

Application Domains of Deep Learning

1. ROBOTIC CONTROL
2. LINEAR SYSTEMS AND NON-LINEAR DYNAMICS
3. DATA MINING
4. AUTONOMOUS NAVIGATION
5. BIOINFORMATICS
6. SPEECH RECOGNITION
7. TEXT ANALYSIS

© Oxford University Press 2021. All rights reserved

Recurrent Neural Networks

SEQUENCE DATA

Some of the sequence data are:

1. Time series data
2. Weather data that is observed over a time period
3. Sequence DNA data as a string of A, C, G and T
4. Sentiment analysis data as a sequence of text with sentiment such as happy or excellent

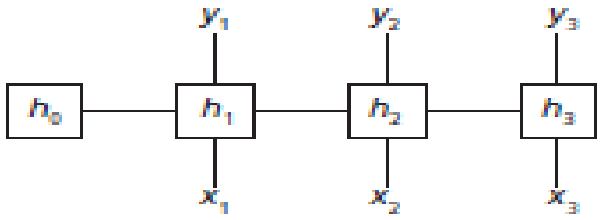


Figure 16.11: A RNN for Three Time Units Sequence Data

© Oxford University Press 2021. All rights reserved

RNN Model

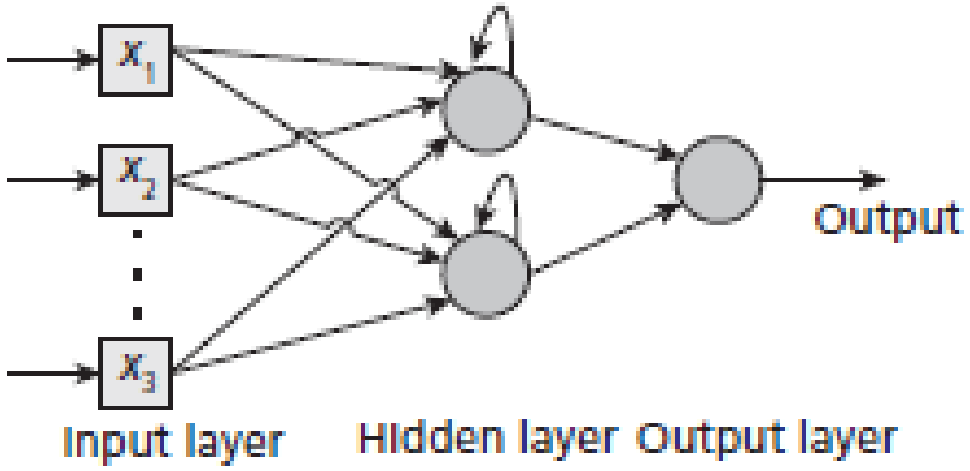


Figure 16.12: Model of Recurrent Neural Network

© Oxford University Press 2021. All rights reserved

Training of RNN

RNN is trained using the backpropagation algorithm as feed forward network. Since RNN has loops, backpropagation does not work in the same manner. So, a new algorithm called Backpropagation through time (BPTT) is used.

The outline of BPTT algorithm is given below.

Algorithm 16.3: BPTT

Step 1: The First step of BPTT algorithm is to unfold RNN.

Step 2: A copy of neurons that contains loops is created and cyclic graphs are converted into acyclic graphs.

Step 3: RNN is converted to feed-forward network.

Step 4: RNN is constructed as a chain of functions and derivatives are computed and chain rule is applied.

© Oxford University Press 2021. All rights reserved

Training of RNN

In the forward pass, the governing equations are implemented as:

$$h_t = \tan h(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

$$y_t = W_{hy} h_t + b_y \tag{16.11}$$

In the backward pass, cross entropy loss function is used. It can be recollected that the cross-entropy loss function is given as:

$$L = -\ln(p_c) \tag{16.12}$$

Here, p_c is the number of correct classifications. The partial derivatives of L, W_{hy} and W_{xh} are obtained in the backward pass and are backpropagated for all time stamps. This is being done in BPTT algorithm. A variant of BPTT algorithm is Truncation Backpropagation Through Time algorithm. It is used as an approximation to RNN that cannot be unfolded.

© Oxford University Press 2021. All rights reserved

Problems of RNN

1. VANISHING GRADIENTS
2. EXPLODING GRADIENTS

© Oxford University Press 2021. All rights reserved

LSTM

VARIANT OF RNN

LONG SHORT-TERM MEMORY

1. Forget gate
2. Input gate
3. Output gate

© Oxford University Press 2021. All rights reserved

Forget Gate

Forget Gate Forget gate is responsible for determining the activations of a cell that need to be forgotten. The input vector x_t is concatenated with the hidden state vector that has been propagated through step h_{t-1} and passes through a sigmoid activation function. The sigmoid function outputs value in the range 0 to 1. The value 0 represents the state of completely removing the information and the value 1 represents the state of keeping the complete information. The output of the sigmoidal function is multiplied with the cell state. Thus, sigmoidal function can be used as a filter to keep the relevant information and get rid of all unnecessary information.

© Oxford University Press 2021. All rights reserved

Input Gate

Input Gate or Update Gate Update gate or Input gate determines when to update and what to update in response to any new input. The input x_t is concatenated with h_{t-1} and passed through a sigmoidal function to generate a vector. The sigmoidal function is called input gate as its output decides which values need to be updated. A value of zero indicates that the values need not be updated and a value of 1 indicates that the values should be updated. The concatenated input also passes through tanh activation function. The tanh function creates another tensor that needs to be added to the state. The outputs of sigmoidal function and tanh function are multiplied and then added to the cell state using vector addition.

© Oxford University Press 2021. All rights reserved

Output Gate

Output Gate Output gate determines what needs to be sent as output. The concatenated input of x_t and h_{t-1} is taken to a sigmoid activation function. The output gate decides what part should be the output. The cell state is sent as input for tanh function. Its value is in the range –1 and +1. Through the outputs of sigmoidal function and tanh function, parts of the output that need to be sent is decided.

© Oxford University Press 2021. All rights reserved

Gated Recurrent Unit (GRU)

There are two gates:

1. Reset Gate
2. Update Gate

Reset Gate Reset gate is used to merge the new input with the previous memory.

Update Gate Update gate determines the amount of previous memory that needs to be merged. It determines how much previous memory is maintained. Thus, update gate is a combination of input and forget gates.

© Oxford University Press 2021. All rights reserved

Summary

1. A neural network that has only one hidden layer is called a shallow network. If a neural network has two or more hidden layers, it is called as deep neural network.
2. Deep neural network extracts features automatically, enabling automatic machine learning.
3. Activation functions create non-linearity. ReLU and Softmax are some of the examples of activation functions.
4. Stochastic Gradient descent (SGD) computes error for every data and adjusts the weights.

© Oxford University Press 2021. All rights reserved

Summary

5. Batch mode updates weight for all training data at once. Mini batch updates the weight after selecting a batch of data.
6. Loss functions measure the error. Some of the examples are mean-square error and cross entropy.
7. Regularization forces to reduce weights. L1 and L2 regularizations are some of the examples of regularization.
8. Dropout drops one of the neurons in the training phase that forces deep nets to learn alternative representations.
9. CNN is a special network that is used for image classification.
10. Some of the advanced architectures are AlexNet, VGG-net, GoogleNet, ResNet and DenseNet. These are useful for transfer learning.
11. RNN is useful for learning from sequence data.
12. Training for RNN is done by using the algorithm, back propagation through time (BPTT).

© Oxford University Press 2021. All rights reserved